

Scalable Technologies for Data Analysis

Storage

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026

Storage

- By the end of this class you should be able to
 - Choose between object storage vs. filesystem vs. database based on cost, operability, and performance
 - Explain why columnar formats (Parquet/ORC) dominate analytics and how compression and pushdown reduce cost
 - Design partitioning that supports common query patterns without exploding partitions
 - Recognize the small files problem and propose mitigations (compaction/merge)
 - Organize a lake with clear conventions (paths, naming, layers, environments)
 - Explain at a high level what lakehouse table formats (Iceberg/Delta/Hudi) add: metadata, snapshots, schema evolution
 - Understand what a catalog and lineage are and why they matter for queries, serving, and cost

The plan

1. Intro to modern Data Platforms and scalability
2. Contract-Driven Design and Data Flow Modeling
3. Storage + table formats 
4. Processing patterns + optimization
5. Orchestration + production reliability
6. Serving: SQL/APIs/semantic layer
7. Quality + observability + cost
8. Incremental + streaming
9. ML integration
10. Governance + evolution



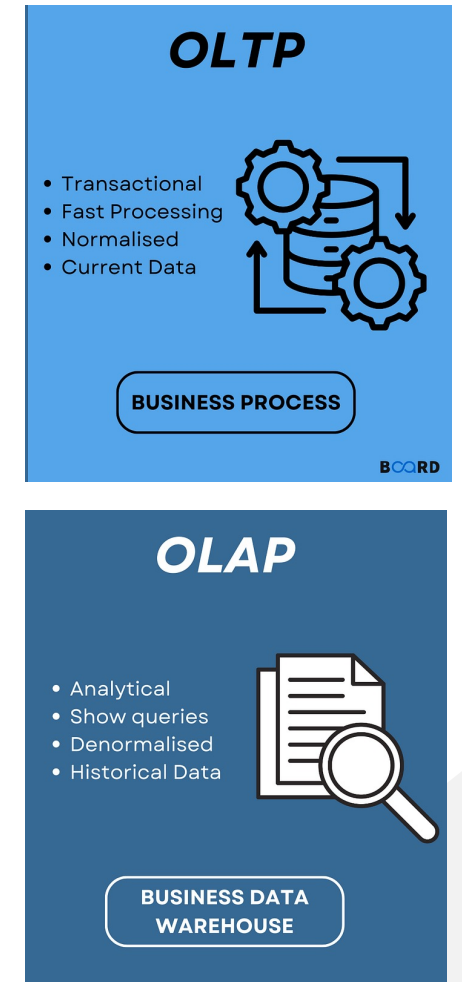
Previously...

- In the previous class, we discussed how to design and model data products, data contracts, DAGs, layers...
- But these artifacts need to be implemented, and live somewhere
- They need to be implemented so that queries are fast and cheap
- Today: **where and how do we store** analytical data at scale so it stays cheap, reliable, and easy to use?

1. The basics

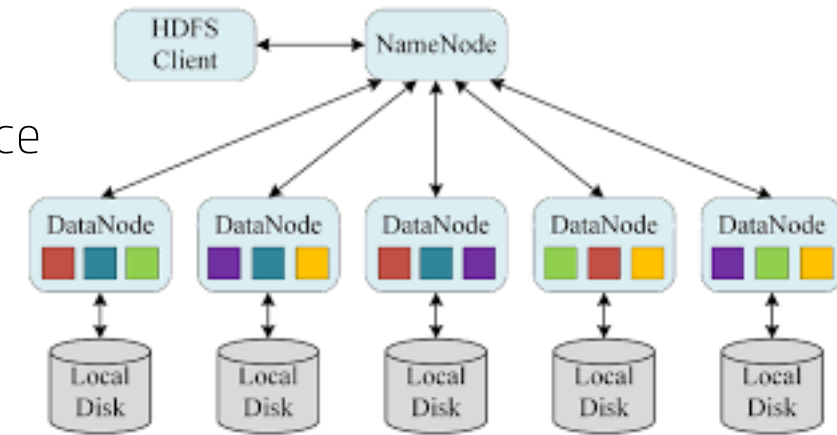
Databases (OLTP / OLAP)

- A system that stores data with table abstractions and often indexes/optimizers
- Two broad families:
 - OLTP: transactions, row-level updates (Postgres, MySQL)
 - OLAP: analytics-optimized engines (column stores, MPP warehouses)
- Strengths
 - **Strong governance** and concurrency built-in (permissions, transactions, constraints)
 - Great serving experience (**low-latency**, predictable performance)
 - Often easier for end users: it's a table, it's fast, it's managed
- Trade-offs
 - Typically **more expensive** per TB than object storage
 - **Less flexible** for raw/semi-structured "everything" storage (especially OLTP)
 - Risk of coupling storage + compute + vendor features (lock-in / operational constraints)
- Best used for
 - Serving layer / curated marts / interactive analytics
 - Workloads needing predictable latency, high concurrency, and strong access controls



Distributed Filesystem (HDFS-style)

- A filesystem distributed across a cluster, presenting a POSIX-like interface
- Designed for high throughput with compute near storage
- Strengths
 - Strong throughput for large sequential reads/writes
 - Filesystem semantics (directories, renames) feel “normal”
 - Historically excellent for large batch systems co-located with compute
- Trade-offs
 - Operationally heavier: you manage a stateful storage cluster
 - Scaling is more involved (capacity and cluster lifecycle are tied together)
 - Less aligned with modern elastic compute patterns (compute/storage separation)
- Best used for
 - Legacy/on-prem big data clusters where compute and storage are tightly coupled
 - Workloads where keeping data “near” compute is a priority



Object Storage (S3 / MinIO)

- A flat key-object store (think “bucket + object name”), not a POSIX filesystem, optimized for durability, cheap scale, not low-latency small operations
- Strengths
 - Cost-effective at large scale (cheap per TB, elastic capacity)
 - High durability and simple scaling (no “resize the cluster” mindset)
 - Enables separation of compute and storage (Trino/Spark/Flyte workers can come and go)
 - Great fit for append-heavy analytical workloads + columnar files (Parquet)
- Trade-offs
 - Many small operations are expensive: listing, lots of tiny files, many partitions
 - “Rename” is often implemented as copy + delete (can be slow/expensive)
 - Latency is higher than local disk, performance depends heavily on layout + file sizes + partitioning
- Best used for
 - Data lakes/lakehouses: Bronze/Silver/Gold tables as files + table metadata
 - Long-term storage, replayability, cheap retention

Question



- If you must store 50 TB of logs, would you pick a DB or object storage? Why?

It depends...

- Primary access pattern
 - Mostly scans/analytics (time-range queries, aggregations, ML feature extraction)
 - Object storage + columnar (Parquet) + table format is usually best
 - Mostly point lookups / interactive filtering (find a specific session/event quickly, lots of small queries)
 - A DB / log search system may fit better
- Latency and concurrency requirements
 - Need sub-second / interactive queries for many users
 - DB/warehouse/search engine often wins.
 - OK with seconds to minutes, batch exploration, scheduled dashboards
 - Object storage is fine (with good partitioning/layout)

It depends...

- Cost model and retention
 - Long retention (months/years), large volume
 - Object storage is typically far cheaper per TB
 - Short retention, high-value hot data only
 - DB might be acceptable
- Update semantics
 - Logs are usually append-only
 - Object storage fits naturally
 - If you need frequent updates/deletes (rare for raw logs, common for GDPR delete workflows)
 - DB/table format with delete support might matter, often handled by curated derived tables rather than raw logs

It depends...

- Query flexibility and schema volatility
 - Logs are messy, semi-structured, evolving
 - Object storage is forgiving (store raw + evolve schemas in derived tables)
 - If you need strict schema and strong constraints at ingestion
 - DB is more natural
- Governance and access controls
 - Fine-grained access, row-level policies, strong auditing
 - Often easier / more natural in DB/warehouse tools
 - Object storage can do governance too, but usually needs more platform pieces (catalog, policies, masking)
- Operational complexity
 - Object storage: simple storage ops, but you must manage layout/partitioning/small files/catalog
 - DB: simpler user experience for querying, but you manage capacity/scale/cost/performance as usage grows

A balanced solution, if usage is not clear

- Store the raw 50 TB in object storage (cheap, durable, scalable)
- Build curated/serving subsets if needed for interactive use-cases in
 - A separate DB/search system
 - This is usually only done for Gold, if there are dashboards/APIs needing low latency
 - Risks having more than one source of truth...
 - Increases complexity
 - Iceberg tables on S3
- Also, frequently
 - Table in Silver contains data from the last 30 days (the “hot window”) for interactive queries
 - Older data stays in object storage only

2. Parquet

And other columnar formats

Formats: row vs. columnar

- Row-oriented (CSV/JSON/Avro)
 - Stores data row by row (all columns together)
 - Efficient when you often read full records
 - Typical use: OLTP-style access, logs as raw text, simple ingestion
- Column-oriented (Parquet/ORC)
 - Stores data column by column
 - Efficient when you read few columns across many rows (classic analytics)
 - Typical use: BI, aggregates, ML feature extraction, large scans

a	b	c
a1	b1	c1
a2	b2	c2
a3	b3	c3

Row-Based Layout

a1	b1	c1	a2	b2	c2	a3	b3	c3
----	----	----	----	----	----	----	----	----

Column-Based Layout

a1	a2	a3	b1	b2	b3	c1	c2	c3
----	----	----	----	----	----	----	----	----

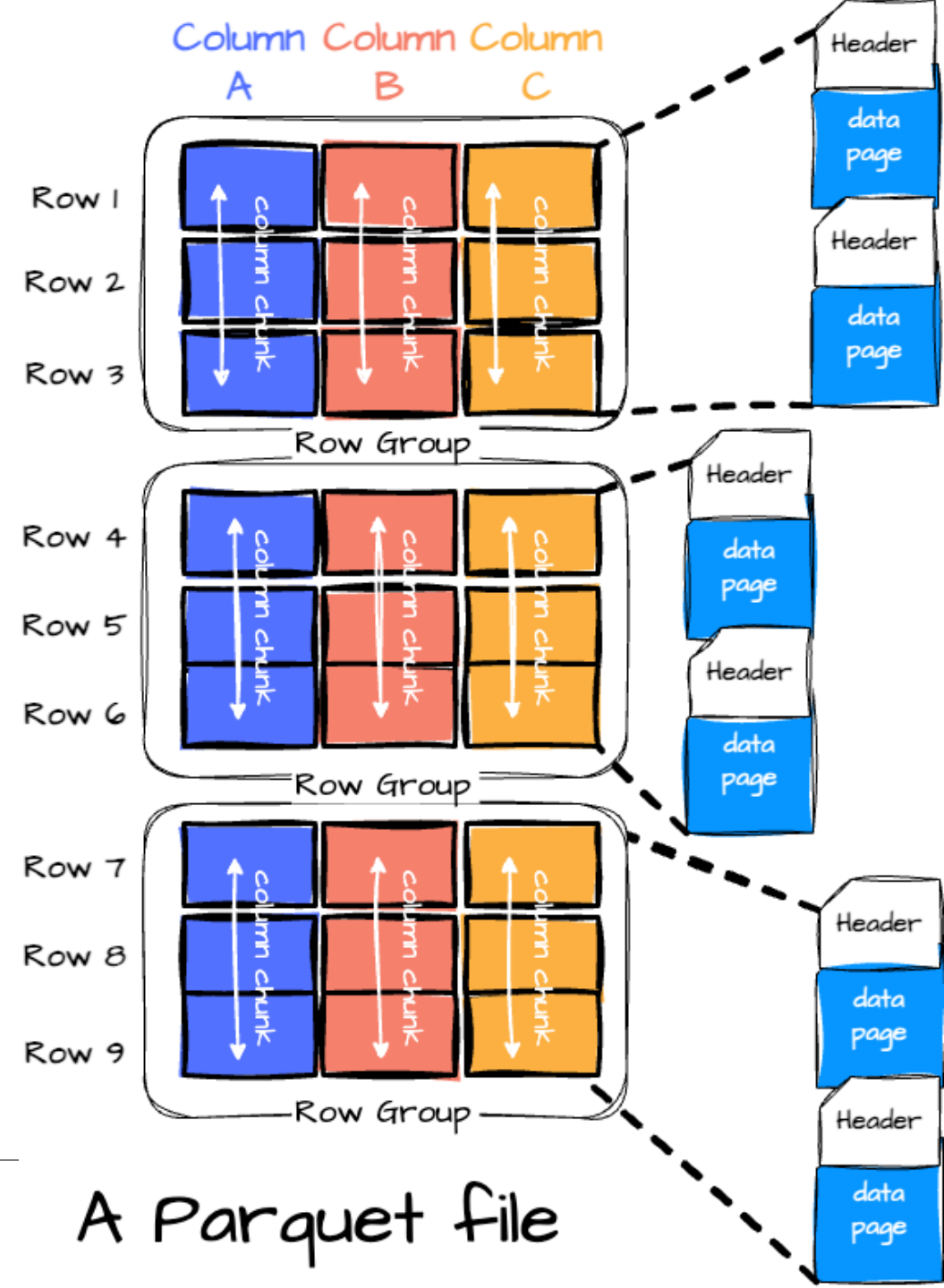
Columnar formats

- Most analytics queries often touch 5–20% of columns but millions of rows, so columnar formats are usually better
- Better compression
 - Similar values are often adjacent (e.g. country codes, event types)
 - Compresses strongly, leads to less I/O, cheaper storage
- Predicate pushdown
 - Engine uses metadata/stats to skip reading chunks that can't match filters
 - Example: WHERE date='2026-02-10' skips row groups whose date min/max is outside
- Projection (read only needed columns)
 - Query only needs `date` and `revenue_eur`, so engine avoids reading 30 other columns
- Parquet
 - A self-described columnar file format that contains all the information needed for the application that consumes the file
 - This allows the software to efficiently understand and process the file without requiring external information
 - The metadata is the crucial part of Parquet



The anatomy of Parquet

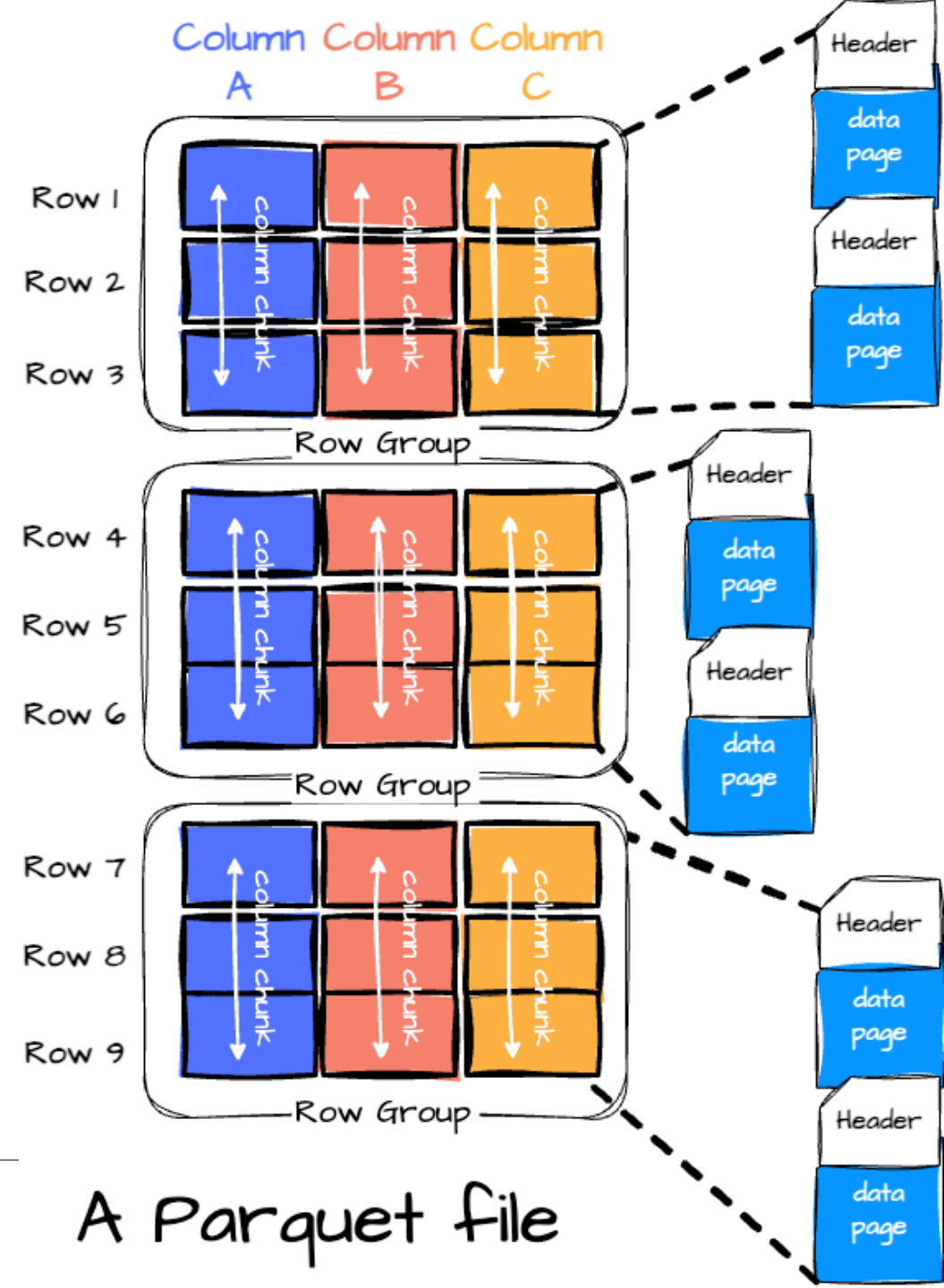
- Row Groups (main chunks)
 - A Parquet file is split into row groups (often ~64-256MB each)
 - A row group contains the same set of rows, but stored column-by-column inside it
 - These are units engines can often skip or read independently
- Column Chunks (per row group)
 - Inside each row group, each column is stored as a column chunk
 - If a query only needs 2 columns, the engine can read only those 2 column chunks (projection)
- Pages + Statistics (within a column chunk)
 - A column chunk is split into pages (smallest data unit). There are pages of different types
 - Data pages (which contain the actual data)
 - Dictionary pages (which contain dictionary-encoded values)
 - Index pages (used for faster data lookup).
 - Pages carry metadata/statistics such as:
 - min / max values (for range filters)
 - null count
 - sometimes distinct count (depending on writer)
 - This metadata enables predicate pushdown: skip pages (and sometimes row groups) that can't match the filter



Row Groups size trade-off

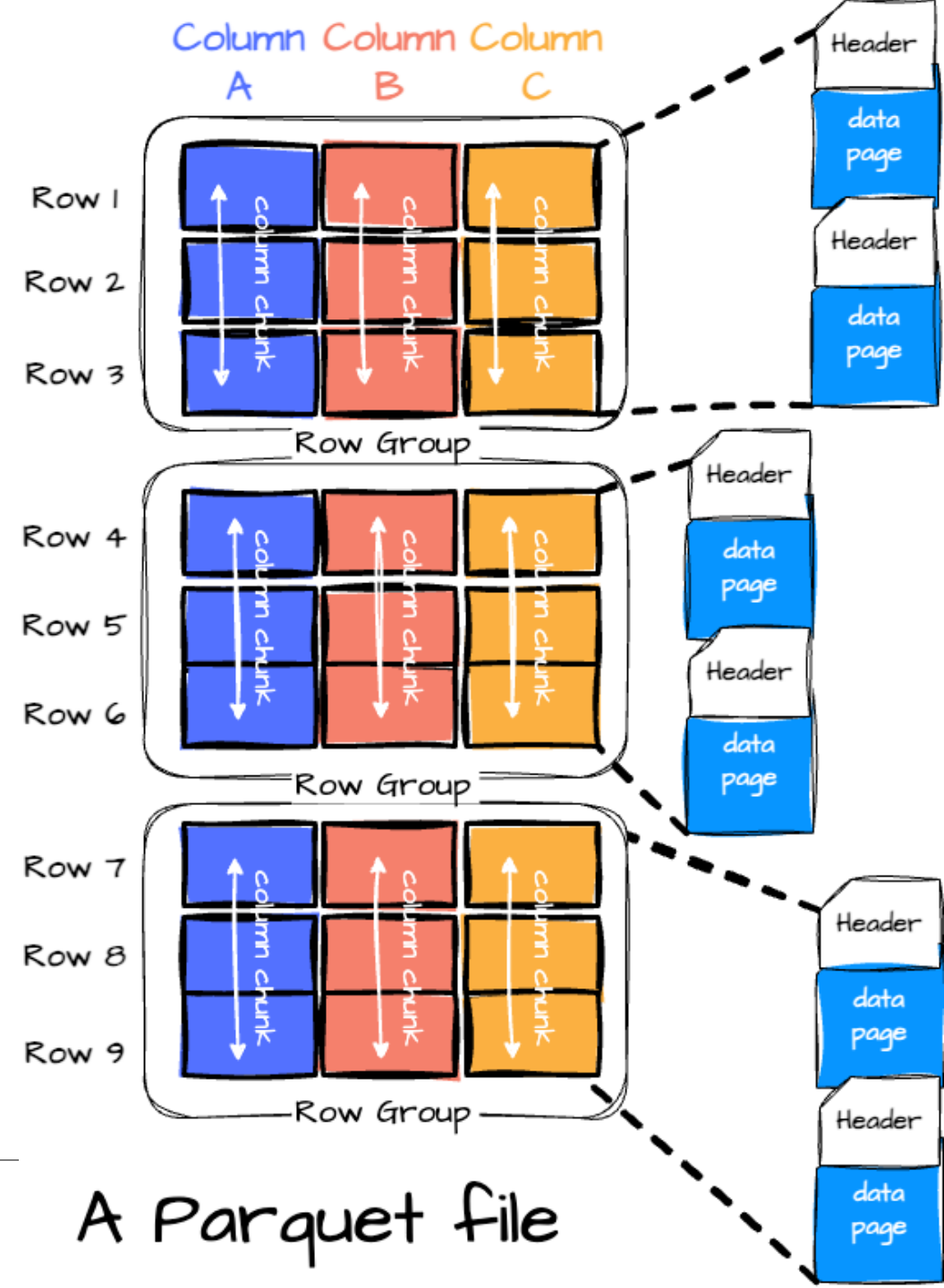
- If row groups are too small...
 - More metadata overhead (more row groups to track)
 - More “tiny work units”, higher planning and per-chunk overhead
 - Can resemble the small files problem, but inside a file
- If row groups are too large...
 - Less effective skipping (filters apply to bigger chunks, fewer can be pruned)
 - Less parallelism: fewer independent chunks for workers to scan
 - Can increase latency for selective queries
- The practical goal is to choose a size that balances:
 - Good scan throughput (sequential reads)
 - Effective **pruning**¹ (row-group/page skipping)
 - Healthy parallelism (enough chunks for distributed reading)
- Row groups relate to (we'll get there soon...)
 - Partitioning (skipping partitions)
 - Sorting/clustering (better stats, better skipping)
 - Compaction (producing healthier files/row groups)

¹When a query engine skips reading parts of a Parquet file (entire row groups or pages) by using stored metadata/statistics



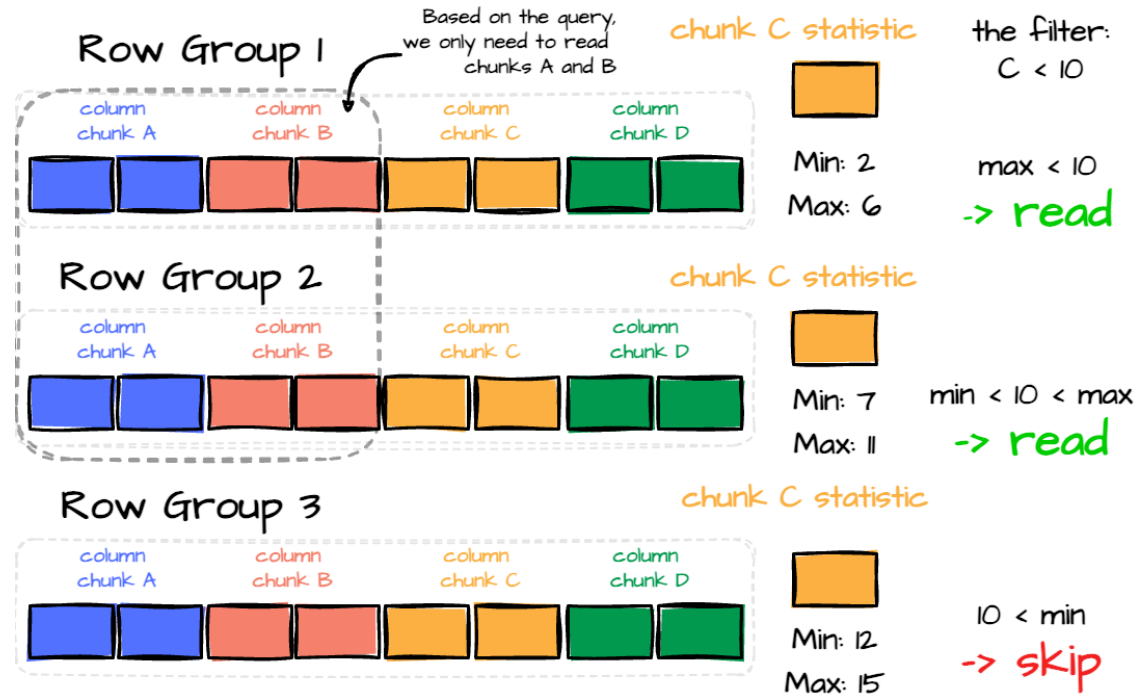
Pruning

- Parquet has standard, optional metadata that writers may include or omit, such as
 - Column statistics (commonly min/max, null count, value count) used for predicate pushdown
 - Dictionary encoding (affects size + speed, indirectly affects what gets stored/how)
 - Page/column indexes (optional structures that help skipping inside files)
 - Bloom filters (optional, helps membership tests for equality filters on high-cardinality columns)
 - How stats are recorded (e.g. truncating long strings, or disabling stats on specific columns)
- Many pipelines allow to configure things like
 - Write stats on/off
 - Write bloom filters on/off
 - Write page indexes on/off
 - Don't write min/max stats for column X (often for **privacy**/sensitivity or to reduce overhead)
- However...
 - We can't invent arbitrary custom page-level metadata beyond what the Parquet format defines
 - We can add file-level key-value metadata, but most query engines won't use that for pruning
- Practical note for our stack (Iceberg + Trino)
 - Even if Parquet stats are limited/disabled, Iceberg also stores its own metadata (file/column metrics in manifests) that engines can use for skipping work
 - **We also influence pruning effectiveness** by how we **sort**/cluster data before writing (better grouping -> more useful min/max)

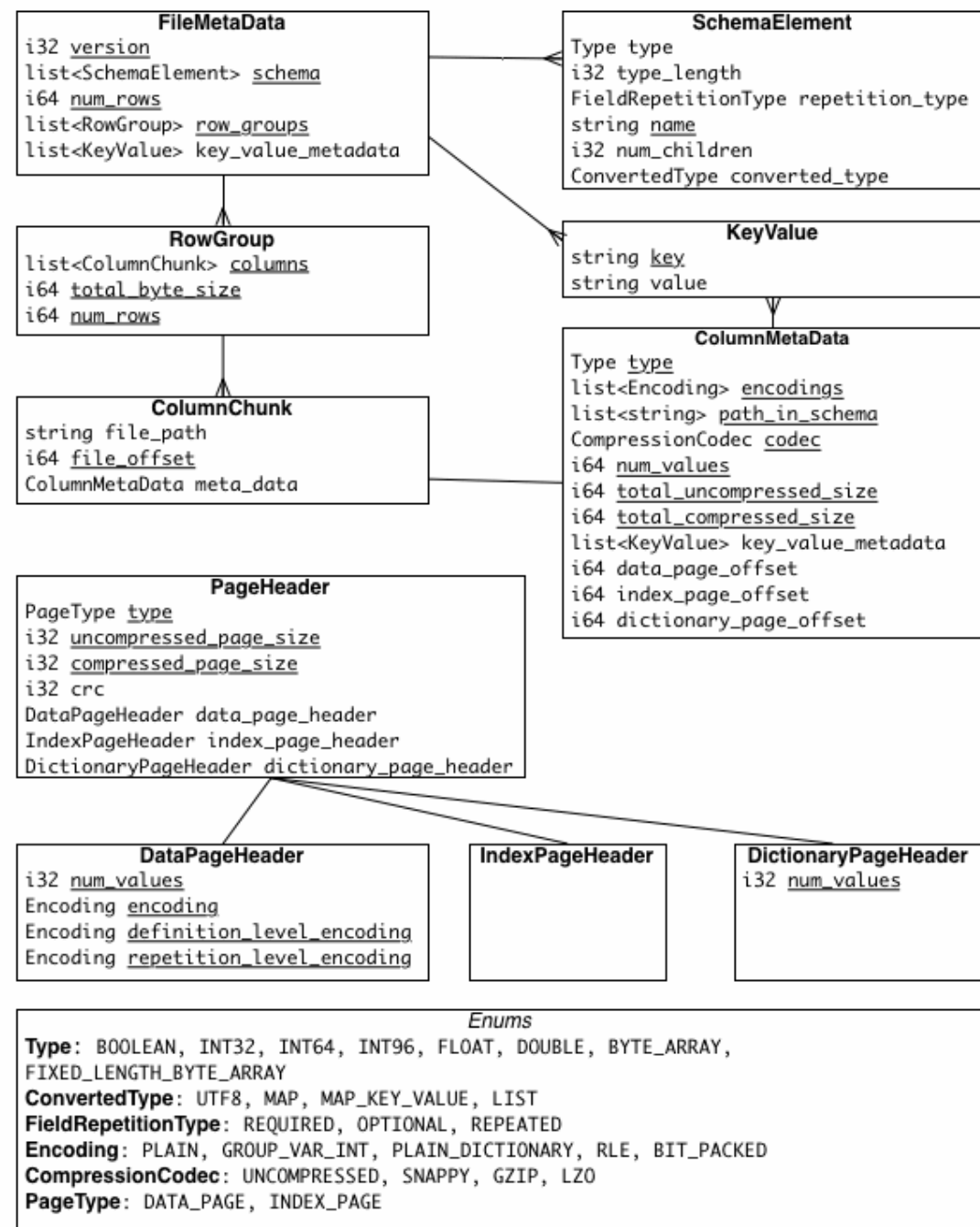


Pruning

```
SELECT column_A, SUM(column_B)
FROM my_table
WHERE column_C < 10
GROUP BY column_A
```

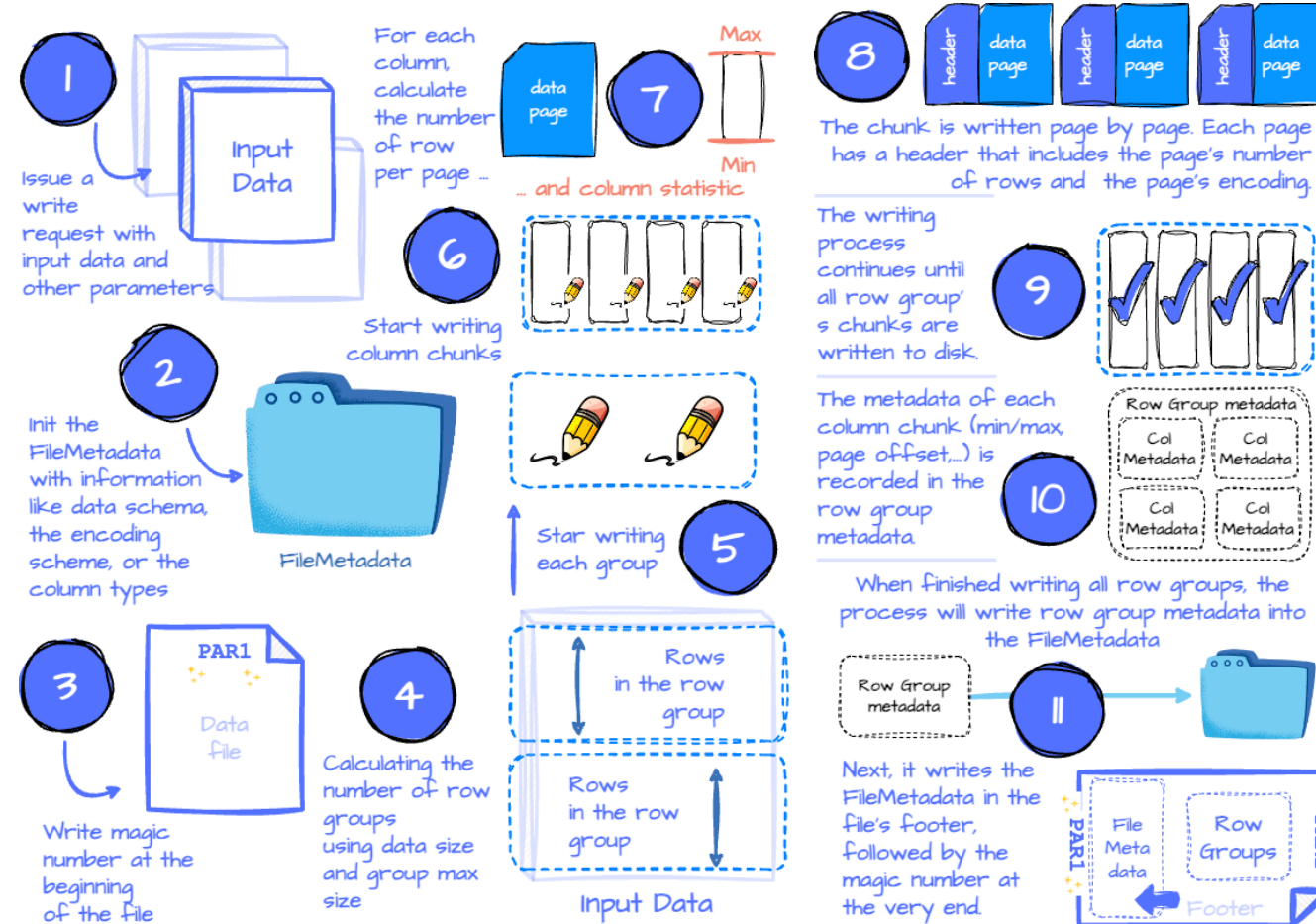


The Parquet data model



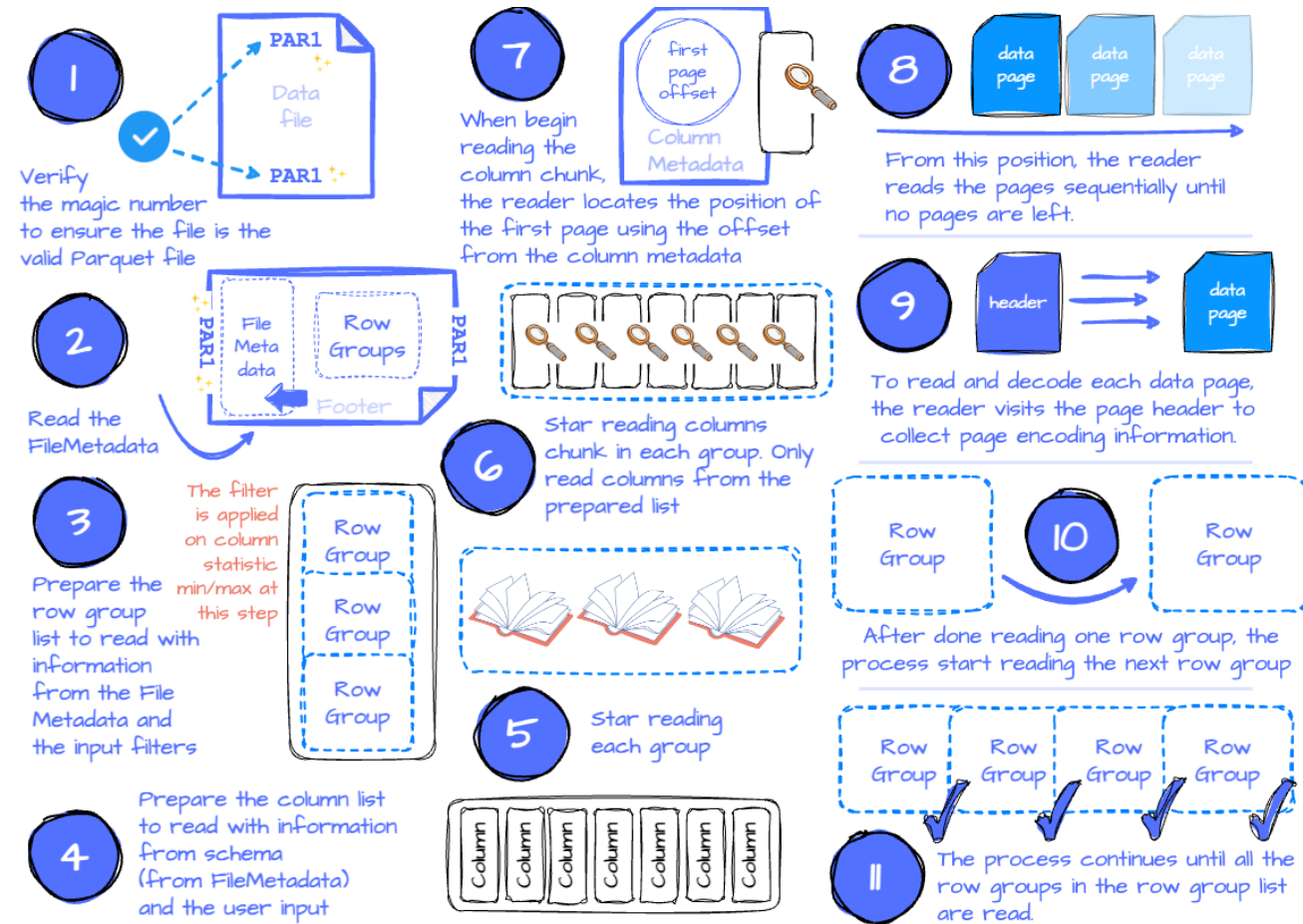
Writing a Parquet file

- Configure writer: schema, optional compression/encoding, optional custom metadata
- Write magic bytes at the start
- Stream rows into a row group buffer until target size
- For each row group:
 - Split into columns, write one column chunk per column
 - Column chunk is a sequence of pages
 - Apply encoding + compression, compute statistics (e.g. min/max, null count) and write statistics pages
 - Record chunk + row group metadata (sizes, offsets, stats)
- Write file footer (schema + row group metadata)
- Write final magic bytes at the end



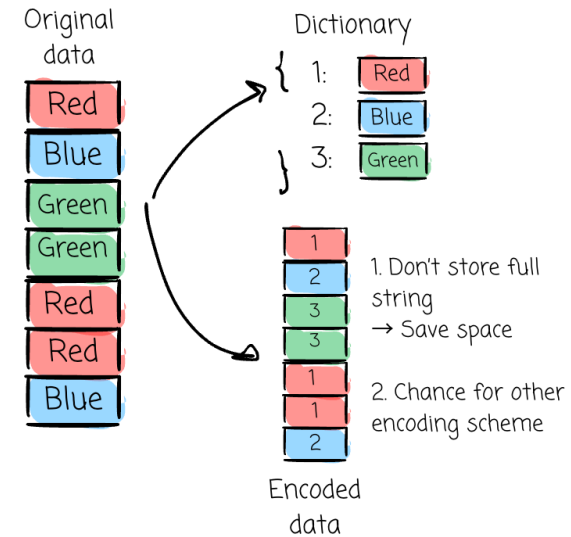
Reading a Parquet file

- Request specifies: file, optional filters, desired columns (projection)
- (Optional) Validate file: check magic bytes (PAR1)
- Read footer metadata: schema + list of row groups + per-column chunk metadata (offsets, sizes, stats)
- Row-group pruning: use stored statistics (e.g., min/max, null count) to skip row groups that can't match filters
- Projection: read only the required column chunks (skip other columns entirely)
- For each selected row group + column
 - Seek to chunk offset (dictionary page first if present)
 - Read pages, decompress + decode using page headers
- Assemble the requested columns/rows -> return results



Encoding

- Encoding is how values are represented as bytes before compression
 - Compression then compresses those encoded bytes, typically page by page
- Common encodings
 - Dictionary encoding: store a dictionary of unique values + small ids (great for categorical columns like country/event_type)
 - RLE / bit-packing: compact representation for repeated/low-cardinality values (booleans, enums)
 - Delta encoding: store differences between consecutive values (good for sorted timestamps/ids)
 - Plain: fallback “as-is” encoding
- Encoding affects file size and scan speed (less I/O, faster decode), and it influences how well compression works



Data

```
data = ['Bruce', 'Bruce', 'Bruce', 'Bruce', 'Bruce', 'Bruce', 'Bruce', 'Bruce', 'Clark', 'Clark', 'Clark', 'Clark', 'Clark']
```

Dictionary Encoding

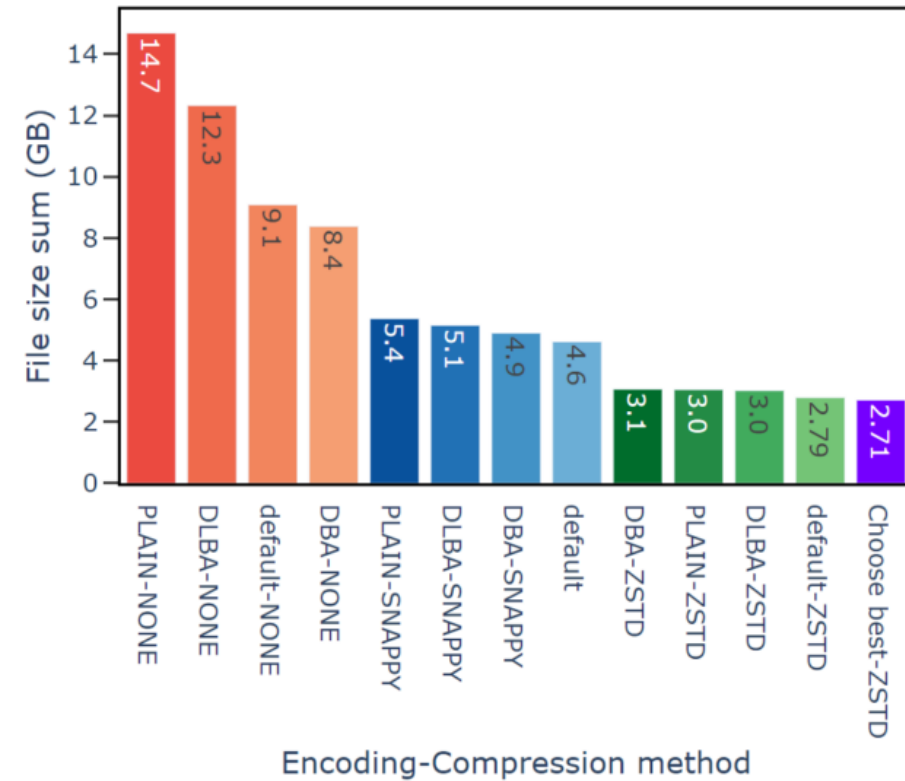
```
dictionary = ['Bruce', 'Clark']  
# replaces repeated values with shorter, unique keys  
indices = [0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
```

RLE Encoding

```
dictionary = ['Bruce', 'Clark']  
indices (RLE) = [(7, 0), (6, 1)]  
# the value 0 appears 7 times consecutive, and value 1 appears 6 times consecutive
```

Compression

- In Parquet, after a column's values are encoded into bytes (per page), Parquet applies a lossless compression codec to those bytes page-by-page (within each column chunk) to reduce the amount of data stored and read
- This means
 - Smaller files, thus cheaper storage
 - Less I/O when reading, thus faster/cheaper queries
- However, there is the trade-off of some extra CPU to compress/ decompress data
- Common codecs include Snappy (default in most writers), ZSTD, Gzip
 - The choice depends on speed vs. compression ratio

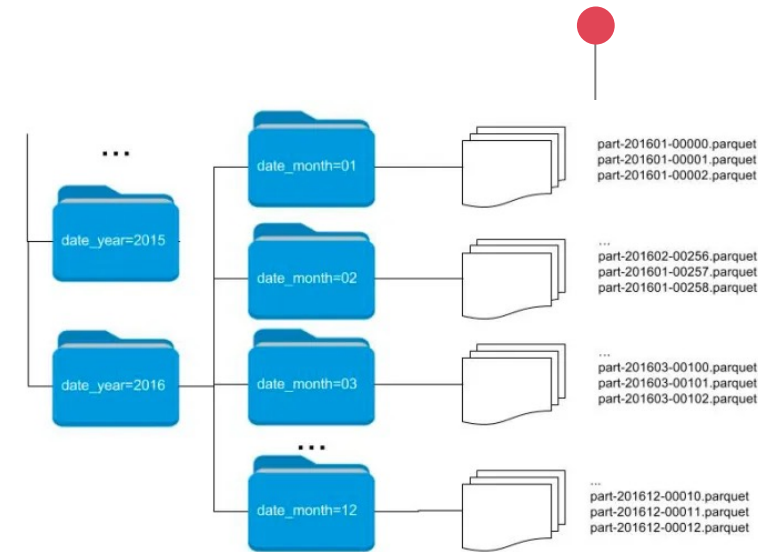


Parallelism

- Parquet enables parallelism at multiple levels:
 - Across files: different workers read different Parquet files concurrently (most common and most scalable)
 - Within a file: read engines can treat row groups as independent scan units: more row groups -> more potential parallel scan
 - Within a row group: engines decode/decompress column chunks/pages in parallel (but this is usually “intra-task” speed rather than distributed parallelism)
- Practical implication
 - Healthy performance depends on having not too few files (limits parallelism) and not too many tiny files (planning/overhead explosion)
 - This connects directly to **partitioning** and the small files problem

Partitioning

- Parquet is a file format, and in a lake a dataset/table is usually many Parquet files, not one big file
- Files are typically organized by partitioned paths
 - Example: .../date=2026-02-10/part-000.parquet
- Each Parquet file is self-describing
 - It has its own footer metadata (schema + row group info + stats)
- Query engines
 - First pick which files to read (often via partition pruning + a catalog/table format)
 - Then read only those files' footers to plan the scan
- Lakehouse table formats such as Iceberg help a lot by tracking exact file lists per snapshot, avoiding expensive “list the folder” patterns and giving consistent reads
- Partitioning is what allows clients to read (as much as possible) **only the necessary** data
 - **Pruning** - the engine skips whole partitions through filters



```
CREATE TABLE users_by_age_band
WITH (
  format = 'PARQUET',
  partitioning = ARRAY['age_band']
)
AS
SELECT * FROM users_unpart;
```

Partitioning

- Contrary to other approaches (e.g. HDFS), Parquet itself doesn't do partitioning
 - Partitioning is a dataset/table-level organization (folders / partition keys in a table format), while Parquet is just the file format inside those partitions
 - We control partitioning in Parquet by **controlling how we write Parquet files** into partitioned layouts (or by using a table format like Iceberg that manages it)
- How to:
 - Choose the partition keys (table/dataset decision)
 - Typical for analytics:
 - Time-based: date / event_date (almost always)
 - Sometimes one additional low-cardinality dimension: country, source, region (careful)
 - Avoid very high cardinality keys: user_id, session_id, as it leads to partition explosion
 - Write data into partitioned paths (classic "Hive-style"), with files placed under folders like:
 - .../events/event_date=2026-02-10/part-000.parquet
 - .../events/event_date=2026-02-11/part-001.parquet
 - Control file sizing within each partition
 - Partitioning creates many buckets, you still need healthy files inside each bucket:
 - Avoid tiny files: tune writer parallelism + compaction
 - Target file sizes (order of 128–512MB)
 - With Iceberg
 - Define a partition spec (e.g. `day(ts)`), and Iceberg tracks which files belong to which partitions in metadata (no reliance on fragile directory listing)
 - Iceberg also supports:
 - Partition evolution (change partition strategy over time)
 - Maintenance operations (rewrite/compaction) to fix small files

In our Stack:

- MinIO stores the Parquet files
- Iceberg defines/maintains partition specs + metadata
- Trino uses Iceberg metadata for partition pruning (and can also benefit from Parquet stats inside files)
- Flyte drives backfills by partition keys (e.g. per day)

Partitioning

- Good partitioning heuristics
 - Partition on fields used in common filters
 - Time is usually best: `event_date`, `date`
 - Avoid extremely high-cardinality keys (`user_id`, `session_id`), which easily leads to millions of partitions, and metadata/planning becomes slow and costly
 - Watch partition size (not tiny, not huge)
- Classic pitfalls
 - Partitioning on a column not used in filters has no benefit
 - Too many partition dimensions leads to combinatorial explosion
 - Timezone confusion leads to inconsistent “days”

Small files problem

- Causes
 - Incremental ingestion producing many tiny outputs
 - Overly fine partitions
 - Too much parallelism in writers
- Symptoms
 - Slow queries from overhead of opening thousands of files
 - Slow planning due to metadata explosion
 - More requests and instability
- In these cases, the system spends more time managing files than processing data
- Practical mitigation
 - Target “healthy” file sizes (rule-of-thumb: ~128–512MB)
 - **Periodic compaction**: rewrite/merge files per partition
 - Avoid partitions with extremely low daily volume (e.g. if after some time we realize we get very little volume per day, we can switch to a monthly key [`year`, `month`])
 - Lakehouse table formats help manage rewrite operations when necessary (Iceberg conceptually)

```
ALTER TABLE users_smallfiles  
EXECUTE optimize(file_size_threshold => '128MB');
```

Lake layout: naming, paths, dataset versioning, layers

- A lake needs to follow conventions to stay sane, especially for larger organizations
- One possible (simple) layout
 - Separate by environment: dev/, prod/
 - Separate by layer: bronze/, silver/, gold/
 - Separate by domain and dataset
 - `s3://lakehouse/prod/silver/ecommerce/events_clean/...`
- Rules of thumb
 - Adhere to stable, readable names (snake_case, avoid spaces)
 - Avoid “random folders”. Everything should have owner/purpose
 - Bronze often append-only; gold is a product (not a dumping ground)
- Versioning note
 - In a lakehouse, versioning is better handled by table snapshots (Iceberg) than by copying “v1/v2/v3” folders
 - Contract/view versioning exists too (refer to previous class)

Tables and metadata

- A lakehouse table is not just files in a folder
- Without table format metadata, we risk fragile schemas, concurrency issues, manual time travel, painful updates
- Iceberg (and similar approaches) add
 - Structured table metadata (what files belong to the table)
 - Snapshots (for time travel) ensures reproducibility and controlled backfills
 - Supported schema evolution (managed change, aligned with the data contract)
 - Writer/reader isolation (readers see consistent state while writers commit)

Catalog

- A data catalog is the system (or service) that gives our data platform a stable naming layer and metadata registry
- At minimum, it should answer
 - What datasets/tables exist?
 - Where are they physically stored?
 - What is the schema / partitioning / table properties?
 - Who owns it and what is it for?
 - How do I discover and trust it?
- What a catalog typically stores
 - Technical metadata
 - Dataset/table name (e.g. `lakehouse.gold.customer_daily_kpis`)
 - Schema, types, partition spec
 - Storage location (bucket/path or table metadata location)
 - Formats (Iceberg/Parquet), properties, snapshots/versions
 - Operational metadata
 - Freshness/quality status, last successful run, SLO compliance
 - Usage signals ("popular tables", query counts)
 - Business metadata
 - Owner/team, description, domain, tags
 - PII classification, governance labels
 - Links to contract/runbook/docs
- Without a catalog
 - You end up with "folder archaeology" and tribal knowledge
 - Breaking changes are harder to detect and communicate
 - Governance is messy (PII, access policies)
 - Engines often have to list directories, which is slow/expensive at scale



Lineage

```
lineage:
  sources:
    - lakehouse.silver.orders_dedup
    - lakehouse.silver.payments_clean
```

```
production:
  orchestrator: flyte
  workflow: kpis.customer_daily_kpis_wf
  schedule: "0 2 * * *" # daily at 02:00
  parameters:
    - data_interval_start
    - data_interval_end
```

```
observability:
  dashboards:
    - pipeline_health: observability.customer_daily_kpis
    - data_quality: quality.customer_daily_kpis
  alerts:
    - type: freshness_breach
      notify: "#data-alerts"
    - type: completeness_breach
      notify: "#data-alerts"
```

- Lineage is the recorded graph of how data was produced and what depends on what
- At minimum, it answers
 - Where did this table come from?
 - What upstream tables/jobs contribute to it?
 - If something changes upstream, what breaks downstream?
- Types of lineage
 - Dataset-level lineage: `gold.customer_daily_kpis` depends on `silver.events_clean`
 - Column-level lineage: `gold.revenue_eur` comes from `silver.order_total_eur` and `silver.refunds`
 - Run-level lineage: which exact run produced this partition/snapshot (time interval, code version, input snapshot)
- Why it matters
 - Impact analysis: "If we change `silver.events_clean`, which Gold products and dashboards are affected?"
 - Debugging: "Why did revenue drop today? Which upstream input changed?"
 - Governance and audit: "Where does this PII field flow to?"
 - Cost attribution: "Which downstream products generate most compute cost?"
 - Reproducibility: (ML especially) "Which dataset snapshot trained this model?"
- In our stack, Flyte is a natural place to capture lineage because it knows the DAG (tasks + dependencies + parameters) (next class...)

Trino: 3-levels namespace

- Catalog – A configured connector to a data source
 - It answers: “Which system am I talking to?”
 - Examples: Iceberg, Hive, Postgres, etc.
- Schema (aka namespace / database) – A logical grouping inside a catalog
 - It answers: “Which database/namespace within that system?”
 - Examples: `iceberg.lab`, `iceberg.tead`
- Table – The named dataset we query
 - It answers: “Which dataset?”
 - Example: `iceberg.lab.users_by_age_band`
- Physically (for Iceberg), a table is a set of data files (Parquet) + Iceberg metadata files, managed as one logical object.
- Fully qualified name: `catalog.schema.table`
 - Example: `iceberg.tead.users_unpart`

The lakehouse way of thinking

- In a lakehouse (such as Iceberg on object storage), data is stored as immutable files (e.g. Parquet). When we do an INSERT, the engine typically
 - Reads the input rows
 - Writes new Parquet file(s) with those rows
 - Commits the new files in Iceberg metadata as a new snapshot
- Contrary to HDFS, it usually does not open and append to an existing Parquet file (until the block is full), because
 - Parquet files aren't designed for safe in-place append: Parquet has a footer with metadata (row groups, offsets, stats). Appending means rewriting/updating the footer and potentially other structures. On object storage, this is awkward and expensive
 - Object storage doesn't support "append" like a filesystem: S3/MinIO objects are typically written as whole objects. "Appending" would mean rewrite the object (copy + replace), which is slow and costly
 - Concurrency and correctness: If multiple jobs try to write to the same file, you get corruption or race conditions. Iceberg avoids this by making writes additive: each writer produces its own files and commits them atomically via metadata.
- Iceberg encourages
 - Many small, immutable files during ingestion, if necessary
 - Then **periodic compaction** (OPTIMIZE command) to merge them into healthier files

Hands-on: understanding the stack so far

- MinIO (object storage)
 - Where the files live (CSV/Parquet + Iceberg metadata)
 - The S3-compatible warehouse
- Parquet (file format)
 - How the data is stored inside files (columnar + compressed, analytics-optimized)
 - Efficient data encoding on disk
- Iceberg (table format)
 - Turns many Parquet files into a managed table with snapshots/time travel, schema evolution, and atomic commits
 - Tables on files
- Hive Metastore (catalog)
 - The table meta-data registry (e.g. name, schema, location, ...)
 - The index
- Trino (query engine)
 - The SQL engine: reads CSV (Hive connector) and reads/writes Iceberg (Iceberg connector)
 - Runs SQL and moves data

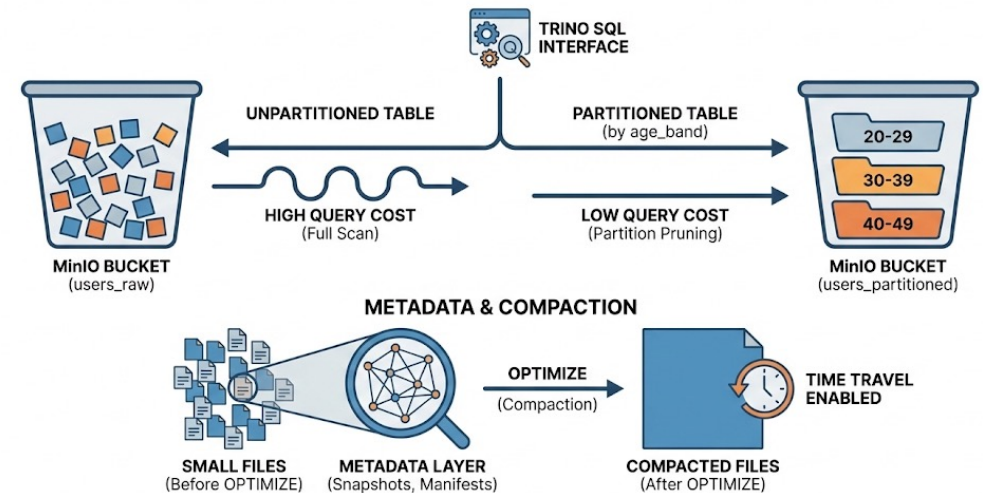
Lifecycle of A SELECT query (read)

- Client submits SQL (CLI/JDBC/BI tool)
- Trino parses and analyzes SQL (syntax, types, permissions)
- Trino asks the relevant connector(s) for table metadata
 - If the table is via Hive connector (CSV/external), connector queries Hive Metastore for schema + location (MinIO path)
 - If the table is via Iceberg connector, connector queries Hive Metastore to get the Iceberg table pointer (where its metadata root is)
- Connector resolves “what files to read”
 - Hive/CSV: list files under the location (and partitions if any)
 - Iceberg: read Iceberg metadata files (snapshot > manifests) in MinIO to get the exact Parquet files in the current snapshot (+ file stats)
- Trino builds a distributed plan (stages > tasks > splits) and schedules work to Workers
- Workers read the data from MinIO (S3 API)
 - CSV reader for raw files, Parquet reader for Parquet
 - Apply projection + predicate pushdown where possible (especially Parquet/Iceberg)
- Workers exchange/shuffle intermediate results (joins/aggregations) as needed
- Trino produces final result and streams it back to the client

Hands-on: managing storage

- In this activity, you will create and explore Iceberg tables stored on MinIO using Trino as the SQL interface. You'll build the same "users" dataset in an unpartitioned table and in a table partitioned by an age_band, then compare how partitioning affects query cost through partition pruning.
- Next, you'll inspect Iceberg metadata (files, partitions, and snapshots) and relate it to the physical objects you see in MinIO.
- Finally, you'll intentionally create a "small files" situation with many small inserts, fix it using Iceberg compaction (OPTIMIZE), and verify the result via file counts and snapshot-based time travel

LAB CONCEPT: ICEBERG TABLE MANAGEMENT & OPTIMIZATION



Scalable Technologies for Data Analysis

Storage

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026